



THE KNIFE

MANUALE TECNICO

Progetto Laboratorio B

Creato da: Mattia Gerundino, Christian Pianarosa

Data: Agosto 2026

Versione: 2.0

SOMMARIO

INTRODUZIONE ED OBIETTIVI DEL PROGETTO	3
SCELTE ARCHITETTURALI	3
Architettura Client-Server Multi-tier	3
Isolamento del Database e Ruolo del Server	4
Gestione Concorrenza e Threading	4
Stack Tecnologico	5
DESIGN PATTERN E SCELTE ALGORITMICHE	5
Design Pattern Applicati	5
Scelte Algoritmiche e Strutture Dati Notevoli	6
Frammenti di Codice Significativi	7
PROGETTAZIONE E MODELLAZIONE UML	8
UML Class Diagram (Struttura Statica ad Alto Livello)	8
UML Sequence Diagram: Flusso di Autenticazione e Sessione	9
UML Sequence Diagram: Ricerca con Cache e Calcolo Distanza	9
UML State Diagram: Ciclo di Vita della Prenotazione	10
PROGETTAZIONE DEL DATABASE (DOCUMENTAZIONE DB ED ER)	10
Analisi dei Requisiti Ristrutturata	10
Schema Entità-Relazione (ER) Concettuale e Ristrutturato	12
Schema Logico Relazionale e Dizionario Dati	13
Vincoli di Integrità e Trigger PL/pgSQL	15
PROTOCOLLO DI COMUNICAZIONE CLIENT-SERVER	16
Struttura Generale dei Messaggi	16
Comandi Pubblici vs Comandi Autenticati	17
Tabella delle Principali Operazioni di Protocollo	17
INSTALLAZIONE E GUIDA OPERATIVA TECNICA	19
Requisiti di Sistema	19
Configurazione Database PostgreSQL	19
Compilazione e Avvio ServerTK	19
Compilazione e Avvio ClientTK	20
DOCUMENTAZIONE DEL CODICE E JAVADOC	20
Standard e Convenzioni Adottate	20
Generazione della Documentazione JavaDoc	20
LIMITI DELLA SOLUZIONE SVILUPPATA ED ESTENSIONI FUTURE	21



SITOGRAFIA/BIBLIOGRAFIA	22
-------------------------------	----



INTRODUZIONE ED OBIETTIVI DEL PROGETTO

TheKnife è un sistema informativo distribuito finalizzato all'esplorazione, valutazione, recensione e prenotazione di ristoranti, ispirato a note piattaforme di discovery culinaria (quali TripAdvisor o TheFork).

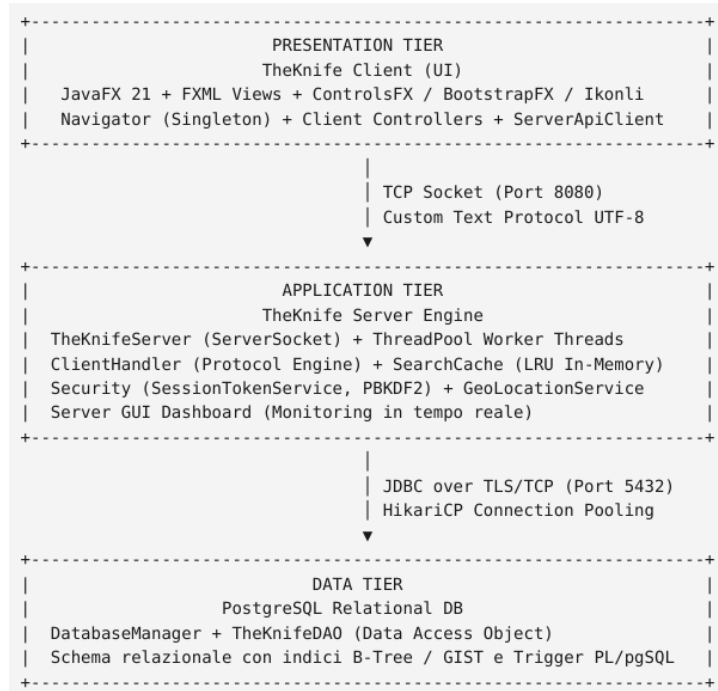
Il progetto è stato sviluppato per il modulo di **Laboratorio Interdisciplinare B** e si prefigge i seguenti traguardi architetturali e funzionali:

- **Separazione rigorosa delle responsabilità:** il client JavaFX non effettua mai connessioni dirette al database relazionale. Tutto il traffico passa attraverso un server dedicato via socket TCP.
- **Supporto multi-ruolo:** fruizione sia da parte di utenti generici/non registrati, sia da parte di *Clienti registrati* (prenotazioni, preferiti, scrittura recensioni), sia da parte di *Ristoratori/Gestori* (gestione anagrafica ristoranti, risposte alle recensioni, monitoraggio KPI).
- **Trasparenza geografica:** geocoding automatico tramite OpenStreetMap (Nominatim API) e calcolo dinamico delle distanze relative dal domicilio dell'utente.
- **Robustezza e sicurezza:** memorizzazione cifrata delle password tramite PBKDF2 salted a 120.000 iterazioni, token di sessione volatili e trigger di integrità a livello DB per impedire privilegi non autorizzati.

SCELTE ARCHITETTURALI

Architettura Client-Server Multi-tier

Il sistema adotta un modello architetturale a tre livelli fisici e logici (3-Tier Architecture):



1. Presentation Tier (Client Desktop):

- Realizzato interamente con JavaFX 21 e FXML.
- Fornisce un'interfaccia reattiva e modulare gestita dal controller di navigazione `Navigator`.
- Comunica esclusivamente via `ServerApiClient` scambiando messaggi testuali UTF-8 su socket TCP.

2. Application / Business Logic Tier (ServerTK):

- Server standalone multi-threaded (disponibile con interfaccia di monitoraggio JavaFX `ServerGuiMain` oppure in modalità headless per container `NonGuiServerMain`).
- Gestisce la validazione dei pacchetti (`RequestValidator`), l'autorizzazione basata su token di sessione temporizzati (`SessionTokenService`), il caching LRU in-memory per le ricerche (`SearchCache`) e il geocoding asincrono (`GeoLocationService`).

3. Data Tier (Database Server):

- Database relazionale PostgreSQL accessibile esclusivamente tramite pool di connessioni ad alte prestazioni `HikariCP` gestito dalla classe `DatabaseManager` e interrogato tramite pattern DAO (`TheKnifeDAO`).

Isolamento del Database e Ruolo del Server

Un requisito progettuale fondamentale è l'assoluto divieto di connessione diretta dal Client al Database.

Vantaggi conseguiti:

- **Protezione delle credenziali del DB:** le credenziali di root/servizio PostgreSQL risiedono solo sulla macchina server e non sono incluse o de-compilabili dal pacchetto client distribuito.
- **Centralizzazione della business logic:** i vincoli applicativi, l'hashing sicuro delle credenziali e le regole sui ruoli sono centralizzati.
- **Ottimizzazione delle risorse:** l'uso del connection pool HikariCP sul server evita l'esaurimento delle connessioni PostgreSQL (connection starvation) consentendo di servire centinaia di client concorrenti con sole 10-20 connessioni fisiche al DB.

Gestione Concorrenza e Threading

- Il server istanzia un `ServerSocket` collegato a un pool a dimensione fissa o controllata di thread tramite `Executors.newFixedThreadPool(maxClients)`.
- A ogni connessione client accettata da `accept()`, viene delegato un worker `ClientHandler` (implementazione di `Runnable`).
- La classe `ClientHandler` mantiene aperto il canale socket per tutta la sessione del client con un read timeout (`SO_TIMEOUT`) di 30 secondi, consentendo chiamate successive senza la latenza di un continuo handshake TCP.
- Tutte le strutture condivise sul server (`ServerStatusRegistry`, `SessionTokenService`, `SearchCache`) sono thread-safe (`ConcurrentHashMap`, locking sincronizzato per le rimozioni LRU).



Stack Tecnologico

- **Linguaggio sorgente:** Java 21 (LTS) sfruttando pattern matching, text blocks (""), record types.
- **GUI Framework:** JavaFX 21 (controls, fxml, graphics, media).
- **Librerie UI:** ControlsFX 11.1.2, BootstrapFX 0.4.0, Ikonli 12.3.1.
- **DBMS:** PostgreSQL 14+ / 16+.
- **Connection Pool:** HikariCP 5.1.0.
- **Driver JDBC:** PostgreSQL JDBC Driver 42.7.2.
- **Geocoding:** Java 11+ Native HttpClient con OpenStreetMap Nominatim API v2.
- **Build System:** Apache Maven 3.9+.

DESIGN PATTERN E SCELTE ALGORITMICHE

Design Pattern Applicati

1. **Data Access Object (DAO) Pattern (*TheKnifeDAO*):**
 - a. Incapsula in modo completo l'accesso al database e tutte le query SQL. I controller e i servizi di logica non scrivono query, ma invocano metodi specializzati quali *cercaRistorante()*, *aggiungiPrenotazione()*, *inserisciRecensione()*.
 - b. Implementa *AutoCloseable* garantendo il rilascio puntuale al termine di ogni blocco di esecuzione in *try-with-resources*.
2. **Singleton Pattern (*Navigator*, *ServerConnectionConfig*):**
 - a. *Navigator.getInstance()*: gestisce il cambio scena su un unico *Stage* JavaFX, il mantenimento dell'ID e del ruolo dell'utente loggato, e la cache dello stato di ricerca evitando flickering o apertura di finestre multiple disordinate.
 - b. *ServerConnectionConfig.getInstance()*: mantiene la configurazione di connessione server/database a livello di istanza unica per il client.
3. **Front Controller / Navigation Mediator Pattern (*Navigator*):**
 - a. Centralizza il routing di tutte le schermate grafiche dell'applicazione client (es. *NotLoggedInUser/home-view.fxml*, *search-view.fxml*, *login-view.fxml*, *Customer/reservations-view.fxml*, *Owner/owner-dashboard-view.fxml*), gestendo passaggi di parametri tra viste diverse in modo trasparente.
4. **Thread Pool / Worker Thread Pattern (*TheKnifeServer*, *ClientHandler*):**
 - a. Disaccoppia l'ascolto delle connessioni socket dalla loro esecuzione effettiva, impedendo a un client lento (slow-client attack) di bloccare l'infrastruttura di ricezione delle altre richieste.
5. **Transfer Object / Data Transfer Object (DTO) Pattern (*RestaurantSummary*, *ReviewItem*, *ApiResponse*):**
 - a. Oggetti serializzabili leggeri utilizzati per incapsulare i dati di ritorno inviati attraverso il protocollo o tra i layer del controller.



Scelte Algoritmiche e Strutture Dati Notevoli

- **Algoritmo di Calcolo Distanza Geografica (Formula dell'Haversine)**

Per determinare i ristoranti nelle vicinanze del domicilio dell'utente senza sovraccaricare il database di estensioni spaziali esterne (o in caso di assenza dell'estensione PostGIS), viene implementata la **formula dell'Haversine** in Java e supportata nel DAO:

$$d = 2R \cdot \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos(\phi_1)\cos(\phi_2)\sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right)$$

dove:

- $R \approx 6371.0$ km (raggio medio della Terra)
- ϕ_1, ϕ_2 sono le latitudini espresse in radianti
- $\Delta\phi = \phi_2 - \phi_1, \Delta\lambda = \lambda_2 - \lambda_1$ (differenze di latitudine e longitudine)

Inoltre, per velocizzare l'esecuzione, il DAO applica un filtro preventivo a **Bounding Box** sfruttando gli indici B-Tree su latitudine e longitudine prima di calcolare l'ordinamento radiometrico:

```
latitudine BETWEEN (? - ?/111.0) AND (? + ?/111.0)
AND longitudine BETWEEN (? - ?/111.0) AND (? + ?/111.0)
```

- **Algoritmo di Caching LRU In-Memory (SearchCache)**

Per alleggerire il carico sul DB per query di ricerca ripetute, il server implementa una cache LRU (Least Recently Used) basata su `LinkedHashMap` con parametro `accessOrder = true`:

- Limite prefissato: `MAX_ENTRIES = 200`.
- Scadenza temporale (TTL): `DEFAULT_TTL_MS = 60000` (60 secondi).
- Il metodo `removeEldestEntry()` elimina automaticamente gli elementi meno consultati una volta superata la capacità.

- **Algoritmo di Sicurezza Password: PBKDF2 con Salt Dinamico**

Le password non sono mai salvate in chiaro né con algoritmi obsoleti (come MD5 o SHA-1 vulnerabili ad attacchi a dizionario e rainbow table).

- Viene utilizzato `PBKDF2WithHmacSHA256`.
- **Iterazioni:** 120.000 round di crittografia.
- **Salt:** 16 byte pseudo-casuali crittograficamente sicuri (`SecureRandom`).
- **Formato salvataggio:** `pbkdf2$<base64_salt>$<base64_hash>$<iterations>`.
- In fase di verifica, viene usata la comparazione in tempo costante (`MessageDigest.isEqual`) per prevenire attacchi di tipo *timing attack*.



Frammenti di Codice Significativi

- Inizializzazione del Pool di Connessioni HikariCP ([DatabaseManager.java](#))

```
public static Connection getConnection() throws SQLException {
    if (dataSource == null) {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl(url);
        config.setUsername(user);
        config.setPassword(password);
        config.setDriverClassName("org.postgresql.Driver");
        config.setMaximumPoolSize(10);
        config.setMinimumIdle(2);
        config.setConnectionTimeout(30000);
        config.setIdleTimeout(600000);
        config.setPoolName("TheKnifePool");
        config.addDataSourceProperty("prepareThreshold", "1");
        config.addDataSourceProperty("preparedStatementCacheQueries", "256");

        dataSource = new HikariDataSource(config);
    }
    return dataSource.getConnection();
}
```

- Generazione Hash PBKDF2 ([PasswordSecurity.java](#))

```
public static String hashPassword(String rawPassword) {
    byte[] salt = new byte[16];
    RANDOM.nextBytes(salt);
    byte[] hash = deriveKey(rawPassword, salt, ITERATIONS, KEY_LENGTH_BITS);

    return PREFIX + "$" +
        Base64.getEncoder().encodeToString(salt) + "$" +
        Base64.getEncoder().encodeToString(hash) + "$" +
        ITERATIONS;
}
```



PROGETTAZIONE E MODELLAZIONE UML

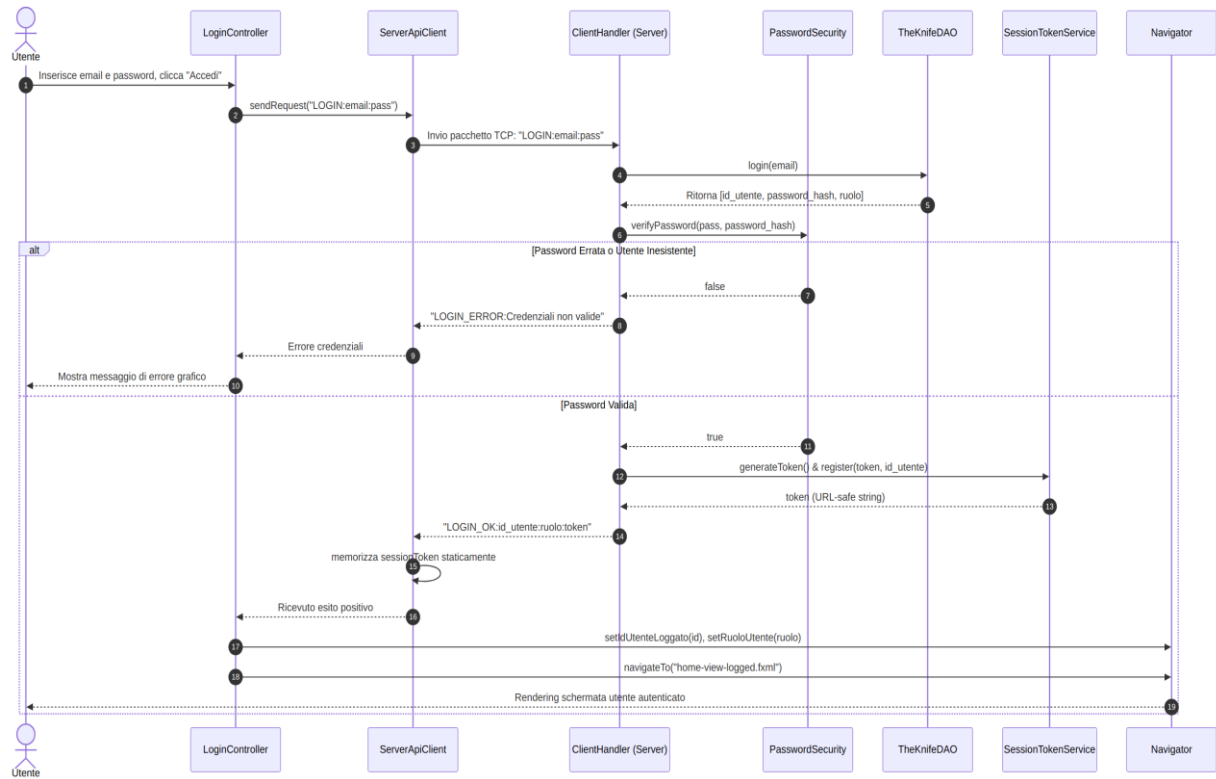
UML Class Diagram (Struttura Statica ad Alto Livello)

Il seguente diagramma descrive la struttura statica ad alto livello del sistema TheKnife, evidenziando la separazione tra il sottosistema Client (UI e gestione viste), il sottosistema Server (elaborazione protocollo, servizi e sicurezza), lo strato DTO/Shared e lo strato di persistenza DAO.



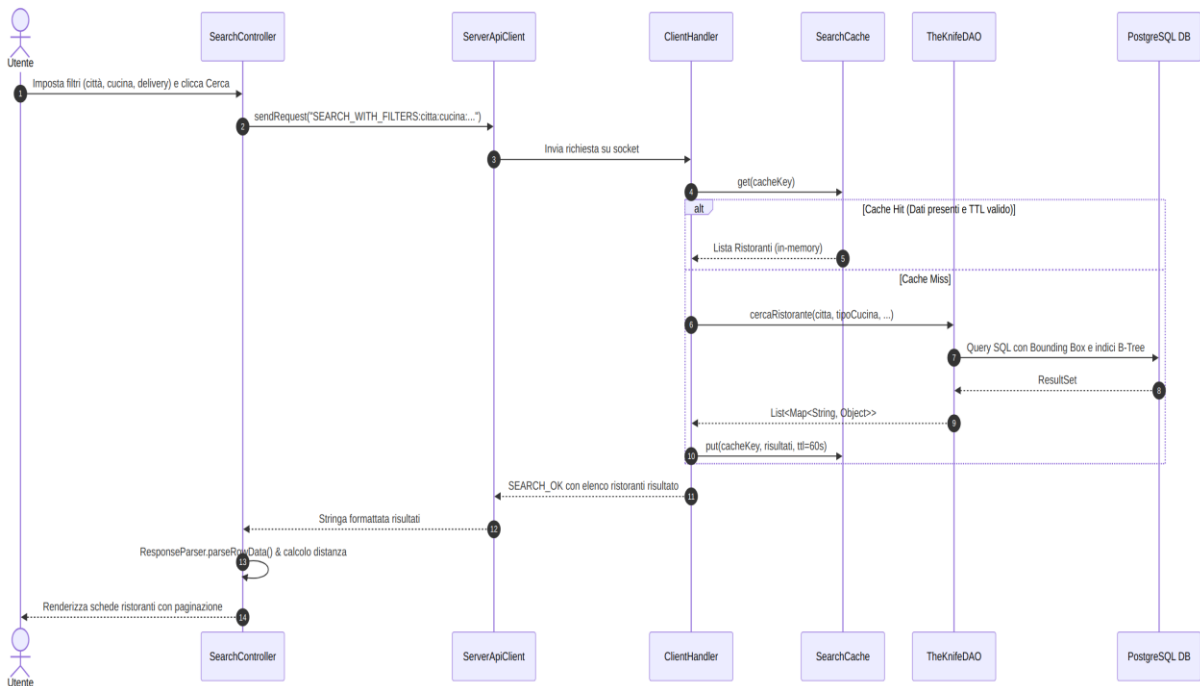
UML Sequence Diagram: Flusso di Autenticazione e Sessione

Il seguente diagramma di sequenza mostra l'interazione dinamica tra Utente, Interfaccia Client, Server e Database durante il processo di login e generazione del token di sessione:



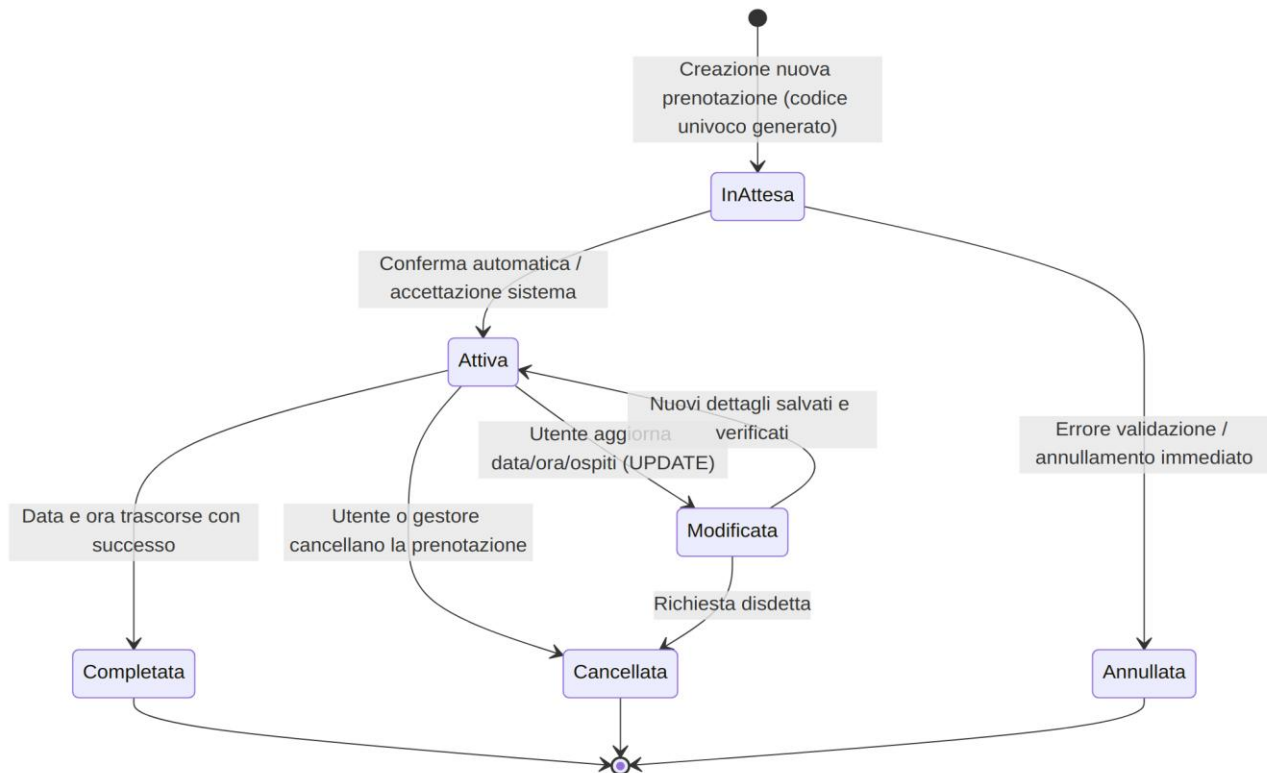
UML Sequence Diagram: Ricerca con Cache e Calcolo Distanza

Questo diagramma cattura la dinamica del sistema durante una richiesta di ricerca ristoranti con filtri, illustrando il controllo del layer di cache e l'esecuzione su DB:



UML State Diagram: Ciclo di Vita della Prenotazione

Lo schema seguente descrive gli stati e le transizioni ammesse per l'entità **Prenotazioni**:



PROGETTAZIONE DEL DATABASE (DOCUMENTAZIONE DB ED ER)

Analisi dei Requisiti Ristrutturata

Specifica Funzionale

Il sistema deve gestire l'anagrafica completa degli utenti del sistema, suddivisi tassativamente in due tipologie mediante vincolo di dominio:

1. **Clienti:** persone fisiche registrate con nome, cognome, e-mail univoca, password cifrata, data di nascita e luogo di domicilio con relative coordinate geografiche (latitudine e longitudine). Possono ricercare locali, inserire recensioni con voto in stelle da 1 a 5, memorizzare ristoranti nei preferiti e creare/modificare/cancellare prenotazioni.
2. **Gestori / Ristoratori:** utenti registrati con gli stessi dati anagrafici, ma abilitati alla proprietà e amministrazione di uno o più ristoranti, alla modifica delle informazioni commerciali e alla pubblicazione di risposte ufficiali alle recensioni dei clienti.

I ristoranti sono caratterizzati da un identificativo alfanumerico univoco (**VARCHAR(50)**, compatibile con gli UUID dei dataset di importazione), denominazione, nazione, città, indirizzo civico completo, coordinate georeferenziate, fascia di prezzo medio, presenza di stelle Michelin o riconoscimenti, obbligo o meno di prenotazione, tipologia culinaria predominante, disponibilità di servizio a domicilio (delivery) e servizio di prenotazione online.



Glossario dei Termini Principali

- **Utente:** persona registrata nel sistema con credenziali univoche e ruolo categorizzato (**cliente** o **gestore**).
- **Ristorante:** esercizio commerciale censito nel database TheKnife, geolocalizzato e associato opzionalmente a un gestore.
- **Recensione:** feedback testuale rilasciato da un cliente autenticato, comprensivo di voto da 1 a 5 stelle. Vale la regola di unicità: un cliente può lasciare al massimo una sola recensione per ogni singolo ristorante.
- **Risposta Recensione:** replica ufficiale redatta dal ristoratore responsabile del locale alla recensione di un cliente.
- **Preferito:** associazione molti-a-molti che consente a un cliente di salvare i propri ristoranti di gradimento.
- **Prenotazione:** richiesta di tavolo per una data e ora prestabilite, per un numero di coperti compreso tra 1 e 20 persone, tracciata da un codice alfanumerico univoco.

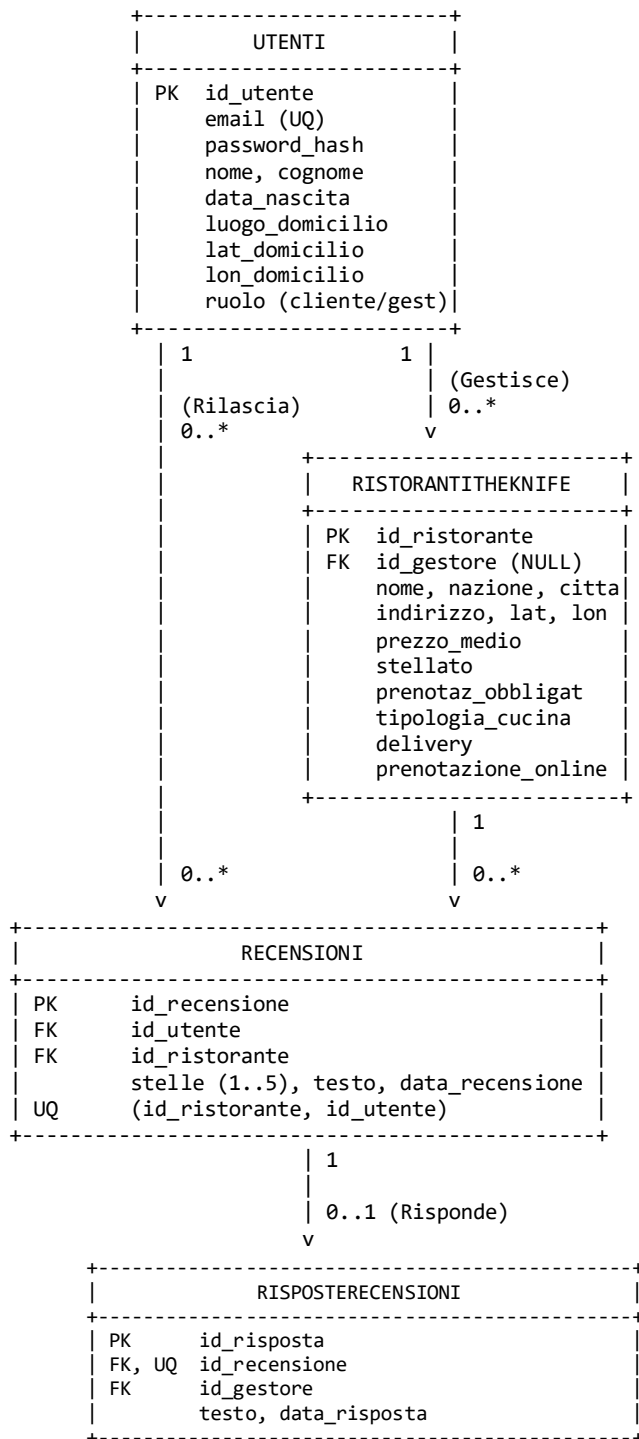
Tavola dei Volumi Stimati e Frequenze d'Accesso

Concetto / Entità	Volume Stimato	Frequenza / Trend Operativo
Utenti	~10.000 record	~50 registrazioni/giorno, molte letture login
RistorantiTheKnife	~3.000 - 50.000 record	Dataset iniziale importato da CSV, letture frequentissime
Recensioni	~100.000 record	Letture continue in visualizzazione dettagli (~500/min in picco)
RisposteRecensioni	~20.000 record	Inserimenti medi, interrogazioni in join con recensioni
Preferiti	~50.000 record	Operazioni frequenti di inserimento/rimozione
Prenotazioni	~80.000 record	Operazioni quotidiane con picchi nei fine settimana



Schema Entità-Relazione (ER) Concettuale e Ristrutturato

Nel processo di ristrutturazione dello schema concettuale sono state eliminate le gerarchie non supportate direttamente dal modello relazionale standard, integrando il ruolo utente tramite discriminatore con vincolo **CHECK** (ruolo IN ('cliente', 'gestore')) e garantendo l'integrità referenziale tramite Foreign Key e Trigger di controllo.



Associazioni Multi-a-Molti e Transazionali:

- PREFERITI (FK id_utente, FK id_ristorante) [PK composta (id_utente, id_ristorante)]
- PRENOTAZIONI (PK id_prenotazione, FK id_utente, FK id_ristorante, codice_prenotazione UQ, data, ora, persone, stato)



Schema Logico Relazionale e Dizionario Dati

- **Tabella Utenti**

Memorizza tutti i soggetti che interagiscono con il sistema con le rispettive credenziali e collocazione geografica.

- **id_utente:** SERIAL PRIMARY KEY — Identificativo univoco progressivo.
- **nome:** VARCHAR(100) NOT NULL — Nome dell'utente.
- **cognome:** VARCHAR(100) NOT NULL — Cognome dell'utente.
- **email:** VARCHAR(255) NOT NULL UNIQUE — Indirizzo email, utilizzato come username univoco per il login.
- **password_hash:** VARCHAR(255) NOT NULL — Hash della password con prefisso crittografico e salt PBKDF2.
- **data_nascita:** DATE NULL — Data di nascita.
- **luogo_domicilio:** VARCHAR(255) NOT NULL — Città o indirizzo di domicilio abituale.
- **lat_domicilio:** DECIMAL(9, 6) NULL — Latitudine calcolata via geocoding (compresa tra -90 e 90).
- **lon_domicilio:** DECIMAL(9, 6) NULL — Longitudine calcolata via geocoding (compresa tra -180 e 180).
- **ruolo:** VARCHAR(10) NOT NULL — Ruolo dell'utente: **cliente** oppure **gestore**.

- **Tabella RistorantiTheKnife**

Catalogo generale delle attività ristorative censite sulla piattaforma.

- **id_ristorante:** VARCHAR(50) PRIMARY KEY — Identificativo primario (stringa alfanumerica o UUID).
- **nome:** VARCHAR(255) NOT NULL — Denominazione commerciale del ristorante.
- **nazione:** VARCHAR(100) NOT NULL — Paese di ubicazione.
- **citta:** VARCHAR(100) NOT NULL — Comune o città del ristorante.
- **indirizzo:** VARCHAR(255) NOT NULL — Indirizzo civico completo.
- **latitudine:** DECIMAL(9, 6) NOT NULL — Coordinate WGS84 di latitudine (-90 .. +90).
- **longitudine:** DECIMAL(9, 6) NOT NULL — Coordinate WGS84 di longitudine (-180 .. +180).
- **prezzo_medio:** INT NOT NULL CHECK (**prezzo_medio** >= 0) — Stima della spesa media a persona in euro.
- **stellato:** INT NOT NULL DEFAULT 0 — Numero di stelle Michelin attribuite (0 per non stellato).
- **prenotazione_obbligatoria:** BOOLEAN NOT NULL DEFAULT FALSE — Flag obbligo di prenotazione anticipata.
- **tipologia_cucina:** VARCHAR(100) NOT NULL — Categoria culinaria principale (es. "Italian", "Japanese").
- **id_gestore:** INT NULL REFERENCES Utenti(**id_utente**) ON DELETE RESTRICT — Riferimento all'utente gestore.
- **delivery:** BOOLEAN NOT NULL DEFAULT FALSE — Disponibilità di servizio a domicilio.
- **prenotazione_online:** BOOLEAN NOT NULL DEFAULT TRUE — Disponibilità del tavolo per prenotazioni da app.

- **Tabella Recensioni**

Valutazioni fornite dai clienti registrati.

- **id_recensione:** SERIAL PRIMARY KEY — Identificativo univoco della recensione.
- **id_ristorante:** VARCHAR(50) NOT NULL REFERENCES RistorantiTheKnife(**id_ristorante**) ON DELETE CASCADE
- **id_utente:** INT NOT NULL REFERENCES Utenti(**id_utente**) ON DELETE CASCADE
- **stelle:** SMALLINT NOT NULL CHECK (**stelle** BETWEEN 1 AND 5) — Punteggio assegnato.
- **testo:** TEXT NOT NULL — Descrizione testuale del feedback del cliente.
- **data_recensione:** TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP — Marca temporale.
- **Vincolo UQ:** UNIQUE (**id_ristorante**, **id_utente**) — Impedisce la duplicazione di recensioni per lo stesso ristorante dallo stesso utente.



- **Tabella RisposteRecensioni**

Repliche ufficiali dei ristoratori alle recensioni.

- id_risposta: SERIAL PRIMARY KEY
- id_recensione: INT NOT NULL UNIQUE REFERENCES Recensioni(id_recensione) ON DELETE CASCADE
- id_gestore: INT NOT NULL REFERENCES Utenti(id_utente) ON DELETE RESTRICT
- testo: TEXT NOT NULL — Testo della risposta.
- data_risposta: TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP

- **Tabella Preferiti**

Tabella ponte per la relazione Many-to-Many tra Utenti e Ristoranti.

- id_utente: INT NOT NULL REFERENCES Utenti(id_utente) ON DELETE CASCADE
- id_ristorante: VARCHAR(50) NOT NULL REFERENCES RistorantiTheKnife(id_ristorante) ON DELETE CASCADE
- **PRIMARY KEY:** (id_utente, id_ristorante)

- **Tabella Prenotazioni**

Storico e prenotazioni attive effettuate dagli utenti.

- id_prenotazione: SERIAL PRIMARY KEY
- id_utente: INT NOT NULL REFERENCES Utenti(id_utente) ON DELETE CASCADE
- id_ristorante: VARCHAR(50) NOT NULL REFERENCES RistorantiTheKnife(id_ristorante) ON DELETE CASCADE
- data_prenotazione: DATE NOT NULL — Data richiesta per il pasto.
- ora_prenotazione: TIME NOT NULL — Orario richiesto.
- numero_persone: INT NOT NULL CHECK (numero_persone BETWEEN 1 AND 20)
- codice_prenotazione: VARCHAR(50) NOT NULL UNIQUE — Codice univoco generato (es. TK-PREN-...).
- note: TEXT NULL — Richieste particolari o intolleranze alimentari.
- stato: VARCHAR(20) NOT NULL DEFAULT 'attiva' — Stato operativo (attiva, cancellata, completata).
- created_at: TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
- updated_at: TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP



Vincoli di Integrità e Trigger PL/pgSQL

Oltre ai classici vincoli dichiarativi (`NOT NULL`, `UNIQUE`, `CHECK`, `FOREIGN KEY ON DELETE CASCADE`), sono stati sviluppati e implementati tre trigger in PL/pgSQL che garantiscono a livello di database l'aderenza tassativa alle regole di business, indipendentemente dall'applicazione client:

- Trigger `tr_check_ruolo_gestore`

Verifica che l'utente assegnato alla colonna `id_gestore` della tabella `RistorantiTheKnife` possieda effettivamente il ruolo di `'gestore'`:

```
CREATE OR REPLACE FUNCTION fn_check_ruolo_gestore()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.id_gestore IS NOT NULL AND (SELECT ruolo FROM Utenti WHERE id_utente = NEW.id_gestore) <> 'gestore' THEN
        RAISE EXCEPTION 'Vincolo violato: l''utente % non ha ruolo ''gestore'', NEW.id_gestore;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER tr_check_ruolo_gestore
BEFORE INSERT OR UPDATE ON RistorantiTheKnife
FOR EACH ROW EXECUTE FUNCTION fn_check_ruolo_gestore();
```

- Trigger `tr_check_ruolo_cliente`

Garantisce che solo un utente con qualifica `'cliente'` possa inserire recensioni, impedendo ai ristoratori di auto-recensire i propri o altri locali:

```
CREATE OR REPLACE FUNCTION fn_check_ruolo_cliente()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF (SELECT ruolo FROM Utenti WHERE id_utente = NEW.id_utente) <> 'cliente' THEN
        RAISE EXCEPTION 'Vincolo violato: l''utente % non ha ruolo ''cliente'', NEW.id_utente;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER tr_check_ruolo_cliente
BEFORE INSERT OR UPDATE ON Recensioni
FOR EACH ROW EXECUTE FUNCTION fn_check_ruolo_cliente();
```



- **Trigger `tr_check_gestore_risposta`**

Assicura che la risposta a una recensione possa essere scritta **esclusivamente** dal ristorante titolare di quello specifico locale recensito:

```
CREATE OR REPLACE FUNCTION fn_check_gestore_risposta()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_gestore_atteso INT;
BEGIN
    SELECT r.id_gestore
    INTO v_gestore_atteso
    FROM RistorantiTheKnife r
    JOIN Recensioni rec ON rec.id_ristorante = r.id_ristorante
    WHERE rec.id_recensione = NEW.id_recensione;

    IF v_gestore_atteso IS DISTINCT FROM NEW.id_gestore THEN
        RAISE EXCEPTION 'Vincolo violato: il gestore % non gestisce il ristorante di questa recensione', NEW.id_gestore;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER tr_check_gestore_risposta
BEFORE INSERT OR UPDATE ON RisposteRecensioni
FOR EACH ROW EXECUTE FUNCTION fn_check_gestore_risposta();
```

Indici per l'Ottimizzazione delle Prestazioni

Sono stati creati indici dedicati B-Tree su:

- `idx_utenti_email`: ricerca immediata all'autenticazione.
- `idx_rist_citta`, `idx_rist_cucina`, `idx_rist_prezzo`, `idx_rist_citta_prezzo`: accelerazione delle ricerche con filtri incrociati.
- `idx_rist_lat_long`: accelerazione del bounding box spaziale.
- `idx_rec_ristorante_stelle`, `idx_rec_utente_data`: calcolo aggregato istantaneo delle medie voto e recupero storico recensioni.
- `idx_prenot_utente_data`, `idx_prenot_ristorante_data`: ordinamento cronologico rapido delle prenotazioni attive e passate.

PROTOCOLLO DI COMUNICAZIONE CLIENT-SERVER

Struttura Generale dei Messaggi

La comunicazione tra client e server avviene su una connessione TCP persistente scambiando stringhe di testo con codifica **UTF-8**, terminate dal carattere di newline `\n`.

Il formato standard di un pacchetto richiesta è:

```
COMANDO:parametro1:parametro2:...:parametroN[:TOKEN:sessionToken]
```

I campi all'interno di ciascun record restituito sono separati da pipe `|` e composti da coppie `chiave=valore`, mentre righe multiple sono delimitate da punto e virgola `;`.



Comandi Pubblici vs Comandi Autenticati

- **Comandi Pubblici:** accessibili senza autenticazione:
 - LOGIN, REGISTRAZIONE, SEARCH, SEARCH_WITH_FILTERS, GET_RESTAURANT_DETAILS, GET_REVIEWS, CHECK_DB_CONNECTION.
- **Comandi Protetti:** richiedono l'allegato :TOKEN:<sessionToken> valido registrato in SessionTokenService. In caso di token non valido o scaduto, il server rigetta la richiesta con:

ERROR:Sessione non valida o scaduta

Tabella delle Principali Operazioni di Protocollo

Comando Richiesto	Parametri Principali	Formato Risposta di Successo	Note Operative
LOGIN	email:password	LOGIN_OK:id_utente:ruolo:token	Genera token di sessione (TTL 30 min)
REGISTRAZIONE	nome:cognome:email:pass:nascita:domicilio:ruolo	REGISTRAZIONE_OK:Utente registrato con successo	Esegue geocoding e hashing PBKDF2
SEARCH_WITH_FILTERS	citta:cucina:prezzoMin:prezzoMax:delivery:prenotaz:minStelle	SEARCH_OK:id=.. nome=..;id=.. nome=..	Consulta la cache LRU prima del DB
SEARCH_NEARBY	latitudine:longitudine:raggioKm:tipo Cucina	SEARCH_OK:id=.. distanza=..;..	Applica Bounding Box e formula Haversine
GET_RESTAURANT_DETAILS	id_ristorante	GET_RESTAURANT_DETAILS_OK:id=.. nome=..	Informazioni complete e statistiche
ADD_PRENOTAZIONE	id_utente:id_ristorante:data:ora:persone:note:TOKEN:t	ADD_PRENOTAZIONE_OK:codice_prenotazione	Genera codice univoco di prenotazione
GET_USER_PRENOTAZIONI	id_utente:TOKEN:t	GET_USER_PRENOTAZIONI_OK:id_prenot=.. ..;..	Ritorna elenco prenotazioni utente



UPDATE_PRENOTAZIONE	id_prenotazione:id_utente:data:ora:persone:note:TOKEN:t	UPDATE_PRENOTAZIONE_OK:Prenotazione aggiornata	Modifica data/ora/ospiti
DELETE_PRENOTAZIONE	id_prenotazione:id_utente:TOKEN:t	DELETE_PRENOTAZIONE_OK:Prenotazione cancellata	Elimina prenotazione attiva
ADD_REVIEW	id_ristorante:id_utente:stelle:testo:TOKEN:t	ADD_REVIEW_OK:Recensione aggiunta con successo	Vincolo: 1 recensione per cliente
ADD_REVIEW_RESPONSE	id_recensione:id_gestore:testo:TOKEN:t	ADD_REVIEW_RESPONSE_OK:Risposta inserita	Trigger verifica proprietà gestore
ADD_PREFERITO	id_utente:id_ristorante:TOKEN:t	ADD_PREFERITO_OK:Aggiunto ai preferiti	Associazione lista preferiti
REMOVE_PREFERITO	id_utente:id_ristorante:TOKEN:t	REMOVE_PREFERITO_OK:Rimosso dai preferiti	Rimozione lista preferiti
ADD_RESTAURANT	nome:nazione:citta:indirizzo:lat:lon:prezzo:cucina:...:TOKEN:t	ADD_RESTAURANT_OK:id_ristorante	Riservato a utenti gestori
GET_OWNER_RESTAURANTS	id_gestore:TOKEN:t	GET_OWNER_RESTAURANTS_OK:id=..\\ nome=..;..	Lista locali appartenenti al gestore



INSTALLAZIONE E GUIDA OPERATIVA TECNICA

Requisiti di Sistema

- **Java Development Kit (JDK):** Versione 21 (LTS) a 64-bit (es. Eclipse Temurin, Oracle JDK).
- **Apache Maven:** Versione 3.8.0 o successiva.
- **Database Server:** PostgreSQL 14, 15 o 16 attivo e in ascolto su porta 5432.
- **Sistema Operativo:** Windows 10/11, macOS (Intel o Apple Silicon), GNU/Linux a 64-bit.
- **Spazio su Disco e RAM:** Minimo 500 MB di spazio libero; 4 GB di RAM consigliati.

Configurazione Database PostgreSQL

1. Assicurarsi che il servizio PostgreSQL sia avviato.
2. Tramite `psql` o interfaccia pgAdmin, creare il database dedicato:

```
CREATE DATABASE theknife;
```

3. L'applicazione integra il modulo `DatabaseInitializer`: al primo avvio del server, esso rileva automaticamente se le tabelle sono presenti; in caso contrario, esegue lo script `/theknife_create_db.sql` incluso nelle risorse e importa il dataset iniziale `ristoranti_clean.csv`.

Compilazione e Avvio ServerTK

- **Modalità A: Server con Dashboard Grafica (JavaFX)**

Consente di monitorare in tempo reale lo stato di PostgreSQL, il numero di client attivi, l'indirizzo IP e le richieste elaborate.

```
cd TheKnifeCompleto-main/PROGETTO
mvn clean compile
mvn exec:java -Dexec.mainClass="theknife.ServerGuiMain"
```

- **Modalità B: Server Headless (Linea di comando senza GUI)**

Ideale per server remoti, ambienti Linux headless o container Docker:

```
mvn clean compile
mvn exec:java -Dexec.mainClass="theknife.NonGuiServerMain" -Dexec.args="--host localhost --port 5432 --db theknife --user postgres --password tuapassword"
```



Compilazione e Avvio ClientTK

Per avviare l'applicazione desktop client:

```
cd TheKnifeCompleto-main/PROGETTO
mvn javafx:run -Djavafx.mainClass="theknife.ClientMain"
```

In alternativa, per l'esecuzione tramite fat-jar o wrapper Launcher:

```
mvn clean package
java -cp target/theknife-app-1.0.0.jar theknife.Launcher
```

DOCUMENTAZIONE DEL CODICE E JAVADOC

Standard e Convenzioni Adottate

Come prescritto dalle linee guida universitarie del corso:

- Ogni package, classe, interfaccia, metodo e attributo significativo è provvisto di commento nel formato predefinito standard `/** ... */`.
- Le classi riportano la descrizione dello scopo generale e il tag `@author` con Cognome, Nome e Matricola:
- Ciascun metodo pubblico o di logica espone:
 - `@param <nome parametro>`: specifica e vincoli del parametro formale.
 - `@return`: descrizione del tipo di dato e del valore calcolato restituito.
 - `@throws <nome eccezione>`: circostanze esatte e cause del sollevamento dell'eccezione.
- Utilizzo dei tag HTML `<code>...</code>`, `<em>...</em>`, `<strong>...</strong>` all'interno dei commenti per arricchire la resa visiva della documentazione generata.

Generazione della Documentazione JavaDoc

Per generare la suite completa di pagine HTML di consultazione tecnica JavaDoc, eseguire il comando Maven nativo:

```
cd TheKnifeCompleto-main/.....
mvn javadoc:javadoc
```

L'output HTML consultabile con qualsiasi browser web verrà generato nella directory:

`target/site/apidocs/index.html`



LIMITI DELLA SOLUZIONE SVILUPPATA ED ESTENSIONI FUTURE

Nonostante la completezza del sistema realizzato, si evidenziano alcuni limiti architetturali e funzionali, oltre a possibili direzioni di evoluzione futura:

1. **Comunicazione via socket non cifrata:** il protocollo di rete utilizza attualmente socket TCP in chiaro. Per un utilizzo in ambienti reali sarebbe opportuno introdurre la cifratura della comunicazione tramite SSL/TLS, ad esempio mediante `SSLSocket` e `SSLServerSocket`, accompagnata dalla gestione e dallo scambio di certificati X.509.
2. **Geocoding e dipendenza da servizi esterni:** la ricerca delle coordinate geografiche si basa sull'endpoint pubblico Nominatim di OpenStreetMap, soggetto a limitazioni sul numero di richieste effettuabili. Per installazioni di produzione o applicazioni su larga scala sarebbe quindi preferibile utilizzare un'istanza locale di Nominatim, adottare un servizio di geocoding dedicato oppure prevedere il salvataggio preventivo delle coordinate geografiche delle vie presenti nel dataset.
3. **Dataset statico:** il dataset utilizzato per i test è statico e non rappresenta necessariamente l'offerta reale, aggiornata e completa dei ristoranti disponibili. Un'evoluzione del sistema potrebbe prevedere l'integrazione con basi di dati o servizi esterni in grado di fornire informazioni aggiornate sui ristoranti, sugli orari e sulla disponibilità.
4. **Infrastruttura necessaria per l'esecuzione:** l'applicazione richiede l'esecuzione di un server locale o raggiungibile tramite rete e la corretta configurazione di un database PostgreSQL. Questo costituisce un limite in termini di semplicità di distribuzione e utilizzo. Una possibile evoluzione potrebbe consistere nell'adozione di un'infrastruttura centralizzata o cloud, con procedure automatizzate di configurazione e distribuzione.
5. **Piattaforma limitata all'ambiente desktop:** l'interfaccia utente è realizzata in JavaFX ed è pertanto pensata principalmente per un ambiente desktop. Non è attualmente disponibile una versione web o mobile. Un'evoluzione significativa potrebbe prevedere la realizzazione di un'interfaccia web responsive o di applicazioni mobile, mantenendo il server come livello centrale per la gestione dei dati e delle operazioni.
6. **Assenza di funzionalità aggiuntive:** il sistema non integra attualmente mappe interattive, sistemi di pagamento online, notifiche tramite e-mail o notifiche push. L'integrazione di tali funzionalità permetterebbe di ampliare le possibilità offerte all'utente e rendere l'applicazione maggiormente adatta a un contesto reale.
7. **Aggiornamento dello stato delle prenotazioni:** lo stato di modifica o conferma di una prenotazione viene attualmente verificato dal client tramite polling o all'apertura della relativa schermata. Per ottenere un aggiornamento in tempo reale sarebbe possibile introdurre un sistema di comunicazione asincrona basato, ad esempio, su WebSocket o Server-Sent Events (SSE), consentendo al server di inviare direttamente al client le variazioni di stato.
8. **Scalabilità:** la capacità del sistema di gestire un numero elevato di utenti simultanei non è stata verificata in un ambiente di produzione. Sarebbero quindi necessari test di carico e di stress per valutare le prestazioni dell'architettura e individuare eventuali colli di bottiglia, introducendo, se necessario, tecniche di caching, gestione concorrente delle richieste e bilanciamento del carico.
9. **Copertura dei test:** i test automatici realizzati coprono principalmente la validazione dei dati e alcuni componenti condivisi, ma non comprendono in modo esaustivo tutti i flussi completi Client-Server né le interazioni con l'interfaccia grafica. Un'evoluzione del progetto potrebbe quindi prevedere una maggiore copertura mediante test di integrazione, test end-to-end e test automatizzati dell'interfaccia utente.



SITOGRAFIA/BIBLIOGRAFIA

Le seguenti risorse sono state consultate per la progettazione, lo sviluppo e la configurazione dell'applicazione:

Sitografia:

- Oracle, **Java Documentation**, <https://docs.oracle.com/en/java/>.
- OpenJFX, **OpenJFX Documentation**, <https://openjfx.io/>.
- Apache Maven, **Maven Documentation**, <https://maven.apache.org/guides/>.
- PostgreSQL Global Development Group, **PostgreSQL Documentation**, <https://www.postgresql.org/docs/>.
- PostgreSQL JDBC, **pgJDBC Documentation**, <https://jdbc.postgresql.org/documentation/>.
- JUnit Team, **JUnit 5 User Guide**, <https://junit.org/junit5/docs/current/user-guide/>.
- HikariCP, **HikariCP Documentation**, <https://github.com/brettwooldridge/HikariCP>.

Bibliografia:

- Herbert Schildt, **Java: The Complete Reference**, 12th Edition, McGraw Hill, 2021.
- Joshua Bloch, **Effective Java**, 3rd Edition, Addison-Wesley Professional, 2018.
- Cay S. Horstmann, **Core Java, Volume I: Fundamentals**, 12th Edition, Pearson, 2021.

